

User Control in Tourism Recommender Systems

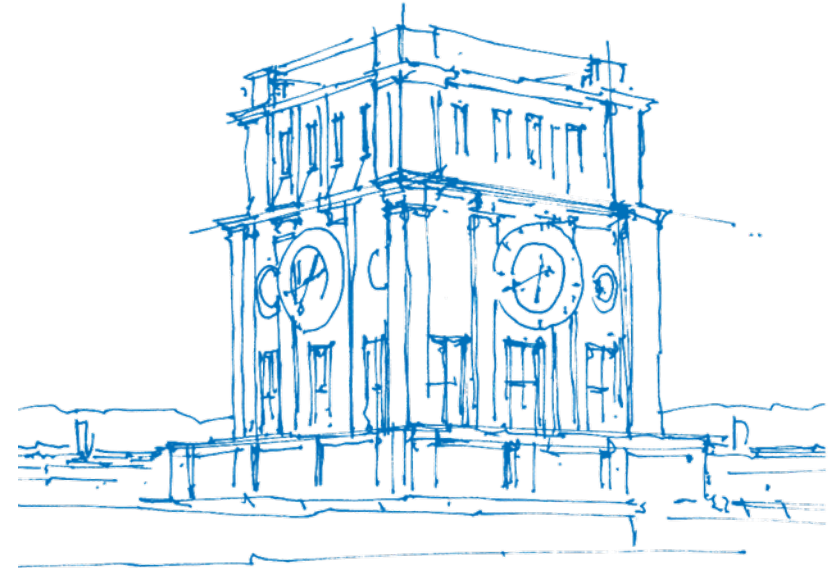
Group W3

Technische Universität München

School of Computation, Information and Technology

Chair of Connected Mobility

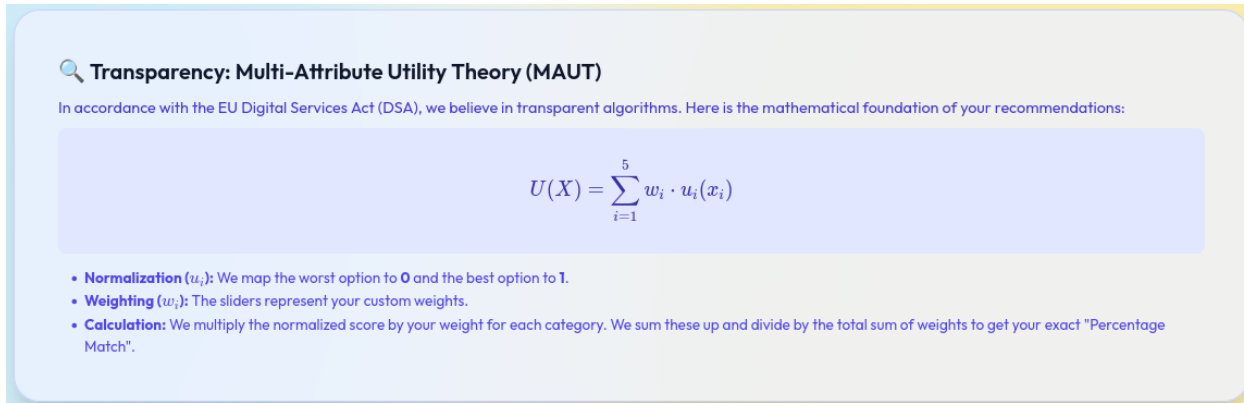
Munich, 20. Juni 2026



TUM Uhrenturm

Last Milestone's Issues

- Issues with calculations (occasional crashes)
- overloaded transparency board
- transparency correct only for MAUT (1st Algorithm)



 **Transparency: Multi-Attribute Utility Theory (MAUT)**

In accordance with the EU Digital Services Act (DSA), we believe in transparent algorithms. Here is the mathematical foundation of your recommendations:

$$U(X) = \sum_{i=1}^5 w_i \cdot u_i(x_i)$$

- **Normalization** (u_i): We map the worst option to 0 and the best option to 1.
- **Weighting** (w_i): The sliders represent your custom weights.
- **Calculation**: We multiply the normalized score by your weight for each category. We sum these up and divide by the total sum of weights to get your exact "Percentage Match".

Abbildung: Overloaded Transparency Panel

What The Finished App Looks Like

Interface Features:

- Correct Transparency panels for MAUT and Constraint-Filtering.
- New Button shows Mathematical Info only on user-intent.
- Transparency panels uses simple language to explain the underlying logic.

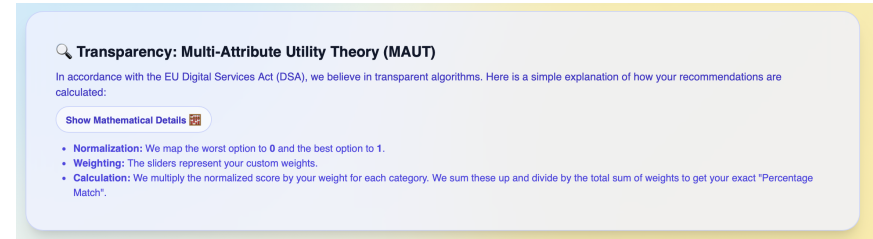


Abbildung: Transparency Board MAUT

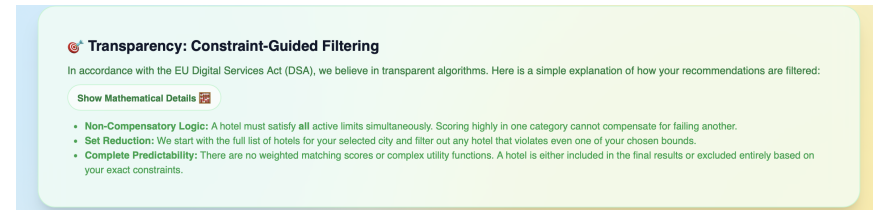


Abbildung: Transparency Board Constraint Filtering

Explanations in simple language

Algorithm A: Weighted MAUT

- **Normalization:** We map the worst option to 0 and the best option to 1.
- **Weighting:** The sliders represent your custom weights.
- **Calculation:** We multiply the normalized score by your weight for each category. We sum these up and divide by the total sum of weights to get your exact PPercentage Match".

Algorithm B: Constraint Filtering

- **Non-Compensatory Logic:** A hotel must satisfy all active limits simultaneously. Scoring highly in one category cannot compensate for failing another.
- **Set Reduction:** We start with the full list of hotels for your selected city and filter out any hotel that violates even one of your chosen bounds.
- **Complete Predictability:** There are no weighted matching scores or complex utility functions. A hotel is either included in the final results or excluded entirely based on your exact constraints.

Demonstration



What's Done & What's Next

- **What has been achieved so far:**

- Built a UI showing real-time updates.
- Implemented transparency abilities to comply with DSA (collapsible breakdown panels, math display).
- Implemented both MAUT and hard constraint filtering modes.
- Fixed all issues (hopefully)

- **Next step & Possible Extensions:**

- **Finish project documentation.**
- **Hybrid mode:** 3rd mode, i.e. Combine both modes (filter out expensive hotels, then rank the rest using MAUT)?
- **Expand mock database:** Add more hotels to the mock database (i.e. 20 each) ?

Thank you for your attention!



Questions?

References I